

Checkpoint 3

¿Cuáles son los tipos de datos en Python?

Entre los tipos de datos que hemos trabajado en el curso se encuentran los siguientes:

- **Booleans:** tienen tan solo dos valores: verdadero (true) o falso (false). Pueden servir para identificar si cierta condición o situación se cumple o no. Por ejemplo, podemos verificar si un post se ha publicado mostrando "true" o "false". Ej. `boolean = True`.
- **Numbers:** números, que se pueden utilizar para cualquier tipo de cálculo. Se distinguen tres tipos de números: integers o números enteros (5,8,12...); floats o números decimales (5.6, 3.78...); complex o números complejos, para cálculos más avanzados ($3 + 3j$).
- **Strings:** es un texto o una secuencia de texto, que deben estar entre comillas, ya sea una, dos o tres comillas. Son inmutables, lo que quiere decir que no se pueden modificar, cada vez que se realiza una operación en un string, lo que sucede es que se produce un nuevo objeto del mismo tipo, en vez de modificar el string existente. Ej. `string = 'Hello, world!'`
- **Lists:** Listas. Una lista que contiene items separados por comas y encerrado entre corchetes. En Python, una lista puede contener items de diferentes tipos de datos. Los valores almacenados en una lista de Python se pueden acceder mediante el operador de división o slice ([] y [:]), con índices que comienzan en 0 al inicio de la lista y avanzan hasta -1 al final.
Ej. `list = [1,2,3,4,5]`
- **Tuples:** es similar a una lista, consiste de un número de valores separados por comas, aunque, esta vez, están cerrados entre paréntesis. Al ser una secuencia, cada item de la secuencia tiene un índice para hacer referencia a su posición en la colección, empezando desde 0.
Ej. `tuple = ('abcd', 786, 2.23, 'john', 70.2)`
- **Bytes y byte arrays:** representa una secuencia de bytes. Cada byte es un valor de número entero entre 0 y 255. Normalmente, se suelen usar para almacenar datos binarios, como imágenes o archivos. Se pueden crear bytes usando la función `bytes()`. En cuanto a los byte arrays, son similares a los bytes, la diferencia es que no son inmutables y se pueden modificar sus valores. Se puede usar la función `bytearray()` para crearlos.
Ej. `b1 = bytes([65, 66, 67, 68, 69])`
Ej. `value = bytearray([72, 101, 108, 108, 111])`
- **None:** Es un tipo especial que representa "nada" o un "valor vacío". Se usa mucho para inicializar variables que recibirán datos más tarde.
- **Sets:** Un set en Python es una colección, pero no es una colección indexada como lo son las strings, lists o tuples. Un objeto no puede aparecer más de una vez en un

set, mientras que en las Listas y Tuplas, el mismo objeto puede aparecer múltiples veces. Los items dentro de un set se separan con comas y se cierran entre llaves. Un set solo puede almacenar objetos que

Ej. set = {123, 452, 5, 6}

- Dictionaries: son una estructura de datos que almacenan datos en pares de clave-valor, permitiendo acceder rápidamente a los valores mediante sus claves, similar a cómo se busca una palabra en un diccionario de la vida real. Cada clave debe ser única e inmutable (como cadenas, números o tuplas), mientras que los valores pueden ser de cualquier tipo y ser modificados. Se definen usando llaves con el formato clave: valor, separados por comas.

Ej. persona = {"nombre": "Ana", "edad": 30, "ciudad": "Madrid"}

¿Qué tipo de convención de nomenclatura deberíamos utilizar para las variables en Python?

Para nombrar las variables en Python, se suele usar el “Snake case”, esto es, se suelen nombrar en minúsculas y, siempre que haya más de una palabra en el nombre, separando las palabras con un guión bajo (_). Nombramos las variables como una_variable y no como UnaVariable, ya que la segunda forma, conocida como Camel case, es como se nombran las clases en Python, y puede llevar a confusión. La guía de estilos oficial de Python, PEP 8, recoge las distintas convenciones de nomenclatura que se usan en el lenguaje. Otra de las convenciones que se usan es el evitar “l”, “O” o “I” como nombres de variables de carácter único, ya que llevan a confusiones.

¿Qué es un Heredoc en Python?

Es una forma de escribir cadenas de texto multilínea sin tener que concatenar strings ni usar muchos saltos de línea manuales. Se realiza utilizando triples comillas al inicio y al final del string.

Ej. heredoc = '''

Esto sería un texto largo

como ejemplo

de un Heredoc

'''

¿Qué es una interpolación de cadenas?

Es una técnica que nos permite insertar valores o expresiones dentro de un texto de manera dinámica, sin tener que concatenar usando + o formateando con múltiples pasos. Al poner f antes de un string, podemos incluir variables entre llaves , {ejemplo}, en el string, los cuales se rellenaran con el valor que hemos establecido previamente. Por ejemplo:

nombre = Ana
cantidad = dos

frase = f 'Hoy, {nombre} se ha comido {cantidad} manzanas.

Las partes {nombre} y {cantidad} se llenarán automáticamente con los valores que se han establecidos previamente (Ana y dos), lo cual hace que la frase sea más dinámica, flexible y fácil de modificar.

¿Cuándo deberíamos usar comentarios en Python?

Hay ciertas razones que sugieren evitar el uso de comentarios a la hora de escribir código. Para empezar, los comentarios suelen ser válidos para el momento en el que se escriben, esto es, si el código se cambia constantemente, a no ser que los comentarios se vayan actualizando, la información que indican queda desactualizada y puede resultar engañosa. Además, si un comentario está mal o es engañoso, puede acabar haciendo más mal que bien. El código debería ser lo suficientemente claro para explicarse por sí mismo, si es necesario comentarlo, es que es demasiado lioso. El uso de comentarios puede acabar derivando en información engañosa, redundante o innecesaria, y puede fomentar malas prácticas.

Ahora bien, también hay ciertos casos donde se pueden usar comentarios. Generalmente, cuando no hay ninguna otra alternativa, o en casos que son realmente difíciles para expresar con código, se puede incluir algún comentario explicativo. También sirven para organizar el código, poniendo titulares a las secciones. Aun así, principalmente, siempre que se vaya a incluir un comentario, es recomendable preguntarse si es necesario dicho comentario y si el código es capaz de expresarse claramente sin él.

¿Cuáles son las diferencias entre aplicaciones monolíticas y de microservicios?

Las aplicaciones monolíticas son aquellas donde el código es un único bloque y todo está unido, backend, frontend, lógica de negocio y acceso a datos todo bien acoplado en un proyecto que se despliega como una sola unidad. Esto hace que al inicio sea más simple de desarrollar y fácil de desplegar. Sin embargo, el código se puede volver difícil de mantener con el tiempo. Al estar todo unido, cualquier pequeño cambio puede afectar a todo el proyecto y resultar en más errores en otras partes del código.

En cuanto a las aplicaciones de microservicios, dividen la aplicación en muchos servicios pequeños independientes, los cuales tienen cada uno su propia responsabilidad, se comunican entre sí y se puede desplegar individualmente. Uno de los mayores beneficios es que son más fáciles de escalar, ya que se pueden escalar las partes independientes en vez de la totalidad. Identificar errores también es más fácil, ya que se pueden realizar pruebas en los diversos componentes y aislar el problema. Además, permiten que varios equipos puedan trabajar en paralelo. Sin embargo, este tipo de aplicaciones requieren un proceso largo y de bastante tiempo, ya que se debe configurar cada servicio de forma independiente. También puede derivar en problemas de compatibilidad entre los servicios cuando se vayan actualizando.